

The domain is the *distraction*. The framework is the answer.

When you don't know the domain, the only thing you have is process. Five wildly different prompts – Uber rides, Netflix ads, food delivery, hotel inventory, ride-hailing surge pricing – each modeled with the same five-move sequence. By drill three, you'll feel the framework taking over.

CONTENTS

01 Why domain doesn't matter (much)
.....

02 The framework, refined
.....

03 The clarification toolkit
.....

04 Drill 1 — Uber rides
.....

05 Drill 2 — Netflix ads
.....

06 Drill 3 — Food delivery
.....

07 Drill 4 — Hotel inventory
.....

08 Drill 5 — Surge pricing
.....

09 Patterns that show up everywhere

10 Wildcard cheat sheet

§ 01 — REFRAME

Why the domain doesn't matter *much*.

A senior data engineer can model any business they've never worked in — that's the actual test. Almost every operational/analytical model decomposes into the same handful of patterns regardless of domain.

THE FACT EVERY DOMAIN HAS

Some **transactional event**. Ride completed. Ad impression. Order delivered. Reservation booked. The fact table is a row per event. Identify it within the first 3 minutes.

THE DIMENSIONS EVERY DOMAIN HAS

Who (user/customer), what (product/content/service), where (geo), when (time), how (channel/device). At least four of these will appear. Sketch the star around the central fact.

THE LIFECYCLE EVERY DOMAIN HAS

Things change state over time. Trip request → match → pickup → in-progress → complete. Order placed → assigned → picked up → delivered. Either model with a state-change fact (one row per transition) or a current-state SCD (one row per entity, current state).

THE MONEY EVERY DOMAIN HAS

Revenue, cost, splits. Driver/host/seller payouts. Multi-party financial flows. Almost every modeling round has a money component – you'll need a payout fact, a price fact, and probably FX rates if it's global.

THE TRAP

Spending the first 5 minutes *showing off* domain knowledge you don't actually have. **Don't** guess what an "ad ops cohort" is or how Uber's surge actually works. Ask. Then model what you've been told. The interviewer is not testing your ad-tech trivia – they're testing whether you can extract requirements from an ambiguous prompt.

§ 02 – FRAMEWORK

The five moves, refined for *unfamiliar* domains.

Same five moves, but the order of emphasis shifts when you're flying blind. Move 1 (clarify) and Move 5 (stress test) get more weight. Moves 2-4 take less time because once you know the grain, the structure follows.

MOVE 1 · 5-7 MINUTES (LONGER THAN USUAL)

Clarify with the right kind of questions.

You're not asking trivia. You're extracting **structural** facts: what's the central event, who consumes the data, what's the SLA, what are the entities, what are the lifecycle stages. See §03 for the toolkit.

MOVE 2 · 2 MINUTES

State the grain — for each fact table.

"One row per X per Y." Out loud, before drawing. If you can't say it cleanly, you don't understand the prompt yet.

MOVE 3 · 8-10 MINUTES

Conceptual model — the star.

Draw the central fact in the middle. Spokes out to dimensions. Name foreign keys. Don't worry about every column — get the shape right.

Mention "I'll fill in attributes once we agree on the structure."

MOVE 4 · 3-5 MINUTES

Physical — partition, cluster, SCD.

Where does this live (Snowflake/Iceberg/Hive)? Partition column?

Cluster keys? Which dims are SCD2? This is the section juniors skip.

MOVE 5 · 5 MINUTES (LONGER THAN USUAL)

Stress test — volunteer 3 failure modes.

This is what makes a Sr/Staff candidate. You're not waiting to be asked. "Three things that worry me about this design: late data, GDPR, and..." The interviewer's eyebrows will go up.

THE SR/STAFF DIFFERENTIATOR

After Move 5, add one more sentence: **"And the data product this enables is..."** — describe the dashboard, alert, ML feature, or operational decision the model unlocks. This shifts you from "person who designs

schemas" to "person who designs systems that drive decisions." That's the level shift Netflix wants to see.

§ 03 — CLARIFICATION TOOLKIT

The seven questions to ask *before* you draw anything.

These work for any domain. Memorize them. In an unfamiliar domain, asking even three of these in the first five minutes signals seniority more than any schema you'll draw.

QUESTION	WHY IT MATTERS
1. Who consumes this and what decisions do they make?	Determines aggregation level, refresh cadence, and which dims you need. "Recommendations team" vs. "finance team" leads to different models.
2. What is the central event?	Locates the fact table. Without this, you'll model the wrong thing. "A ride," "an impression," "an order" — pin it down.
3. What's the freshness SLA — minutes, hours, or days?	Drives batch vs. streaming, and the partitioning scheme. Real-time means columnar storage matters less; cheap appends matter more.
4. What's the retention requirement?	30 days vs. 7 years. Affects whether you keep raw events or only aggregates. Drives storage cost conversations.
5. What are the	State machine. "What happens between request and

lifecycle stages of the central entity?	completion?" Surfaces the state-change fact you might otherwise miss.
6. Are the natural keys stable, or do they get reassigned?	Critical for SCD design. A user_id that's recycled needs different handling from one that's permanent.
7. Is this global or single-region? Multi-currency?	Drives time zone handling, FX rates as a dim, country-specific dimensions, GDPR boundaries.

Things to volunteer (without being asked)

- "I'm going to assume X. Tell me if that's wrong." – explicit assumptions are gold. Silence is junior.
- "The grain of this fact is one row per [X] per [Y]." – said out loud, before drawing.
- "There are two failure modes I'd want to address up front: [late data] and [schema evolution]."
- "This unlocks the following data products / decisions: [name 1-2]."

§ 04 – DRILL 1

Uber rides — operations & payouts.

DRILL · 01

MARKETPLACE · 25 MIN

"Design a data model for Uber's ride business. Stakeholders include the operations team (efficiency, ETA accuracy,

cancellations), the finance team (driver payouts, take rate, taxes), and the city ops team (which markets to expand or pull back from). The data should support both real-time monitoring and historical analytics."

► WALK-THROUGH

§ 05 — DRILL 2

Netflix ads — impressions, attribution, billing.

DRILL · 02

ADTECH · 25 MIN

"Design a data model for Netflix's ad-supported tier. The system must support: ad serving (deciding which ad to show), attribution (which ad drove a signup or purchase), billing (advertisers pay per impression/completion/click), and reporting (advertisers want to see their campaign performance daily)."

► WALK-THROUGH

§ 06 — DRILL 3

Food delivery — three-sided marketplace.

"Design a data model for a DoorDash-style food delivery service. Three sides: customers placing orders, restaurants fulfilling them, drivers (Dashers) delivering them. The data should support operations (on-time rate, customer support), restaurant performance reporting, driver payouts, and customer-facing order tracking."

► WALK-THROUGH

§ 07 — DRILL 4

Hotel inventory — the temporal modeling test.

"Design a data model for a hotel booking platform like Booking.com. The system must support: search (showing available rooms with prices for any future date range), reservations (booking, modifying, cancelling), revenue management for hotel owners (occupancy %, ADR, RevPAR), and pricing experiments."

► WALK-THROUGH

§ 08 — DRILL 5

Surge pricing — the temporal-spatial test.

DRILL · 05

TEMPORAL · SPATIAL · 25 MIN

"Design a data model for a surge pricing system at a ride-hailing or delivery company. The model must support: real-time decisioning (what surge multiplier applies to a given location right now?), historical analysis (when and where does surge happen, and what's the effect on supply/demand?), regulatory audit (prove that surge was applied correctly for any specific trip), and ML feature engineering for the next-gen pricing model."

► WALK-THROUGH

§ 09 — PATTERNS

The patterns that *show up everywhere*.

Across all 5 drills, the same handful of patterns recur. Internalize these and any modeling prompt becomes a re-application.

PATTERN	HOW TO RECOGNIZE · HOW TO MODEL
State machine	Anything with a lifecycle (trip, order, booking, content). Two facts: a state-change log (one row per transition, append-only) and a current-state view (latest row per entity). Don't pick one — model both.
Dual-grain fact	One transaction (booking) but analytics on smaller

	units (night). Solution: a transaction fact + a derived line-grain fact. Document derivation rule.
Multi-currency money	Always store local + USD + fx_rate + currency_code. Never just USD. Auditors and finance will demand the local truth.
Time zones	Store TIMESTAMP_TZ in UTC + a derived local_business_date . Finance reports always need local; analytics often want both.
SCD2 on dimensions	Anything that changes over time and where history matters. Driver tier, hotel rate plan, ad creative version, talent contract. effective_from / effective_to / is_current .
Append-only audit log	For anything regulatory or financial. Surge decisions, trip state changes, deal amendments. Never UPDATE; always INSERT.
Tokenized PII	For ad attribution, multi-tenant analytics, GDPR. Replace user_id with token at the edge; keep mapping in a separate, access-controlled service.
Multi-party financial flow	Three-sided marketplaces have payouts going both ways. Separate facts for each direction (rider charge, driver payout, advertiser bill, restaurant payout). Don't conflate.
Spatial cell aggregation	Geo-bounded operations use H3 hexes. Cell_id as a dim key. Resolution is a configuration choice – state it.
Decision audit trail	Any algorithm that affects pricing/billing must be reproducible. Log inputs, outputs, model_version. Immutable.
Late-arriving facts	Almost universal. UPSERT/MERGE on natural key

	with a configurable reprocessing window (typically 7 days).
Idempotent loads	Re-running yesterday's batch produces the same result. MERGE on natural key. Mention this on every drill.
Bridge for many-to-many	One delivery, multiple orders (stacked). One overall deal, multiple titles. Don't denormalize into either side; bridge table.
Real-time vs analytical split	Streaming serving layer (Redis/Cassandra/Druid) for sub-100ms; analytical layer (Iceberg/Snowflake) for batch. Different optimizations. Mention which queries go where.

§ 10 – WILDCARD CHEATSHEET

The page for any modeling prompt.

The opening 30 seconds (out loud, every time)

1. "Let me restate to make sure I understand: you want to model [X] to support [Y consumers] for [Z decisions]."
2. "Before I start drawing, can I ask a few clarifying questions?"
3. Ask 3-5 from the seven-question toolkit (§03).
4. State assumptions: "I'll assume [A], [B], [C] – let me know if any of those need adjustment."
5. "OK, let me sketch the central fact and grain."

The seven clarifying questions (memorize)

1. Who consumes this and what decisions?
2. What's the central event?
3. Freshness SLA?
4. Retention?
5. Lifecycle stages of the entity?
6. Are natural keys stable?
7. Multi-region / multi-currency?

The default fact tables to consider

IF DOMAIN HAS...	YOU PROBABLY NEED...
A transaction (ride, order, impression, booking)	<code>fact_transaction</code> at transaction grain
A lifecycle / state machine	<code>fact_state_change</code> append-only log
Money flowing to multiple parties	Separate <code>fact_payout_*</code> per party
Pricing decisions	<code>fact_price_quote</code> or <code>fact_decision</code> for audit
Aggregable behavior over time	<code>fact_*_snapshot_daily</code> for cheap rollups
Many-to-many relationship	<code>bridge_X_Y</code>

The five things to volunteer in Move 5 (stress test)

1. **Late-arriving facts** – and how the model handles them (MERGE, reprocessing window).
2. **Time zone / multi-currency / multi-region** – local vs UTC, FX rate dim, regional storage.
3. **Schema evolution** – registry, versioned events, permissive raw layer.
4. **PII / GDPR / data residency** – tokenization, access tiers, delete cascade.
5. **Idempotency & reprocessing** – natural keys, MERGE, deterministic outputs.

Sentences to keep ready

- "Let me state the grain explicitly: one row per [X] per [Y]."
- "I'm going to make these dimensions SCD2 because [A, B, C] all change and history matters for analytics."
- "Three things that worry me about this design: [1, 2, 3]. Let me address each."
- "The data product this enables is [a dashboard / an alert / an ML feature / a decision support tool] for [stakeholder]."
- "In production I'd parameterize [X] rather than hardcoding."
- "There's a real-time serving path and an analytical path; this model is for analytical. Let me note where the streaming layer would diverge."

THE BIGGEST MISTAKE

Drawing tables before clarifying. Even if the prompt feels obvious, the act of asking questions changes the interviewer's perception of you from "junior who jumps to code" to "senior who gathers requirements." Spend 5



minutes asking. The remaining 20 minutes will go faster because you know what you're building.

Wildcard pack · the domain is the distraction · same five moves, every time.